Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
«Реализация спроектированной системы»

по курсу «Основы разработки безопасных цифровых систем»

Работу выполнил студент группы №5140901/61501
_____ Чикилев М.А.

Работу принял преподаватель
_____ Арушанян А.В.

“10” ноября 2026 года

Санкт-Петербург
2026

Репозиторий

https://github.com/basketcase7/pgas

Реестр изменений по сравнению с ЛР № 2

Ниже перечислено всё, в чём реализованная система отличается от проекта ЛР № 2, и причина каждого изменения. Остальные решения ЛР № 2 (контракты OP-01…OP-05, схема данных, роли, точки пересечения границ) реализованы без изменений. Тот же реестр лежит в репозитории: `docs/discrepancies.md`.

Таблица 1 – Реестр изменений

№	В проекте ЛР № 2	В реализации	Причина
D-01	Пароли ролей сервисов – литералы CHANGE_ME_* в schema.sql; роль svc_llm	Блок ролей вынесен в db/roles.sql, пароли подставляет Liquibase из окружения; svc_llm не создаётся	Секреты не хранятся в репозитории; llm-orchestrator не разворачивается по условию ЛР3
D-02	Брокер RabbitMQ + DLQ (AMQP)	Apache Kafka 3.9 (KRaft), SASL/PLAIN + ACL, топики *.DLT	Хранимый журнал событий, порядок по ключу заявки, независимые группы потребителей (раздел 2.2)
D-03	llm-orchestrator, OP-06/OP-11, кэш отчётов модели, B-10…B-12	OP-06 и OP-11 → 501 Not Implemented, контейнер не разворачивается	Исключение, прямо указанное в задании ЛР3
D-04	Объектное хранилище (MinIO)	SeaweedFS 3.97 с S3-совместимым API	Образы MinIO больше не публикуются в Docker Hub; S3-контракт сохранён
D-05	B-03: HTTPS PUT по presigned-ссылке напрямую в хранилище	Presigned POST с политикой (ключ, тип, точный размер, 15 мин) через reverse-proxy nginx веб-контейнера; порт хранилища не публикуется	PUT не позволяет хранилищу проверить размер/тип; хранилище не отдаёт CORS. Файл по-прежнему не проходит через API
D-06	TLS на внутренних связях (SQL/TLS, RESP/TLS, AMQP/TLS, LDAPS, SMTP/TLS)	TLS на внешней границе (шлюз 8443, nginx→шлюз с проверкой сертификата); внутри сетей Docker – без TLS, аутентификация сторон сохранена	Локальное развёртывание на одном хосте
D-07	OP-02 принимает declared_grounds[], в схеме нет столбца	Миграция 001: тип support_ground, столбец applications.declared_grounds, CHECK; поле возвращается в OP-01	Данные контракта иначе терялись бы; сотруднику нужны основания
D-08	У svc_applications нет чтения application_scores, protocols	GRANT SELECT на эти таблицы	OP-01 отдаёт scores[]
D-09	К Redis подключены auth, review, llm	Также api-gateway (rl:*), applications (чтение session:*,	Отзыв токена во всех сервисах; счётчики лимитов заявлены в ЛР2

№	В проекте ЛР № 2	В реализации	Причина
		enum404:*), notifier (notifier:*), у каждого свой ACL	
D-10	–	Тестовые пароли: маркер !bootstrap в seed.sql, хеш вычисляет auth-service при старте	Готовый хеш потребовал бы хранить перец в репозитории
D-11	Нет операции получения кампаний	OP-17 GET /campaigns	Клиенту нужны campaign_id и критерии
D-12	Нет перехода draft → submitted	OP-18 POST /applications/{id}/submit	Заявка иначе не попадает на рассмотрение
D-13	OP-09 без описания; нет регистрации и выхода	OP-09 в два шага (запрос + confirm), OP-14 register, OP-15 logout	Требования ЛР3 п. 3.1–3.3
D-14	OP-05: повторный ввод пароля – механизм не описан	OP-16 POST /auth/reauth (отметка в сессии Redis, окно 5 мин), иначе 403 reauth_required	Реализация требования контракта
D-15	Откуда notifier берёт адрес – не описано	recipient_id в событии; адрес – GET /internal/users/{id}/contact у auth-service по сервисному токену	ПДн не попадают в брокер
D-16	В брокер публикует только applications-service	Также review-service (смена статуса) и auth-service (письмо восстановления)	Статус меняет review-service; почту отправляет только notifier
D-17	С каталогом R-09 связан только auth-service	applications-service читает LDAP (только чтение)	OP-02: статус обучающегося – на момент подачи
D-18	B-15: подписанная выгрузка в бухгалтерию	Интеграция не реализуется; ответ OP-05 содержит SHA-256; registry_id = protocol_id	R-10 вне системы; отдельной таблицы реестров нет
D-19	Коды для ряда ситуаций не заданы	OP-05 чужая комиссия → 403; OP-02 неизвестная кампания → 400; ключ с другим телом → 422; неверный UUID → 404	Уточнение контракта
D-20	Политика токенов не детализирована	JWT RS256 10 мин в памяти; refresh в cookie HttpOnly/Secure/SameSite=Strict, сессия 8 ч, ротация	Уточнение политики (раздел 5)
D-21	Лимит для OP-07 не задан	Вход 20/мин с IP, регистрация 5/ч, восстановление 10/ч	Защита от перебора
D-22	–	Служебные контейнеры liquibase, kafka-init, kafka-ui; заглушки ldap (R-09), mailpit (R-08)	Миграции, ACL, демонстрация; локальные заглушки внешних систем

Репозиторий: [ссылка на git-репозиторий – вставить после публикации, проверить доступ]. Локальная копия репозитория с историей коммитов находится в каталоге `lab3`.

Содержание

1. Цель работы 5
2. Исходные данные и технологии 6
2.1. Используемые технологии 6
2.2. Обоснование замены RabbitMQ на Kafka 6
3. Задание 1. Проект и окружение 8
3.1. Репозиторий и структура 8
3.2. Запуск одной командой 8
3.3. Переменные окружения и секреты 9
3.4. README 10
4. Задание 2. База данных 11
4.1. Схема и миграции 11
4.2. Учётные записи приложения 11
4.3. Тестовые данные 12
5. Задание 3. Аутентификация и токены 13
5.1. Регистрация и вход 13
5.2. Политика токенов 13
5.3. Восстановление доступа 14
6. Задание 4. Эндпоинты по контракту 15
6.1. Объём 15
6.2. Соответствие контракту 15
6.3. Порядок вызовов 15
6.4. Повторная отправка 16
6.5. Обработка ошибок 16
7. Задание 5. Межсервисное взаимодействие 18
7.1. Брокер Kafka 18
7.2. Redis 20
7.3. Синхронные вызовы 20
8. Задание 6. Клиентское приложение 21
9. Задание 7. Проверка и карта кода 24
9.1. Коллекция запросов 24
9.2. Три проверочных запроса 24
9.3. Карта кода 25
10. Пошаговое выполнение работы 27
11. Выводы 29

1. Цель работы

Реализовать и развернуть локально систему, спроектированную в ЛР № 2 (сервис заявок на материальную поддержку и ПГАС, вариант 5), так, чтобы каждое проектное решение было выражено в работающем коде: контракты стали обработчиками, DDL-скрипт – базой данных, блок решений по безопасности – проверками в коде.

Результат работы: репозиторий с исходным кодом шести сервисов и двух клиентских приложений, конфигурация Docker Compose, поднимающая 15 контейнеров одной командой, миграции Liquibase, коллекция запросов с автоматической проверкой (149 проверок, все успешны) и настоящий отчёт.

2. Исходные данные и технологии

Техническое задание – отчёт по ЛР № 2: контейнерная диаграмма, реестр точек пересечения границ B-01…B-16, контракты OP-01…OP-06, schema.sql и диаграммы последовательности. Интеграция с языковой моделью по условию ЛР № 3 не реализуется (OP-06/OP-11 отвечают 501).

2.1. Используемые технологии

Таблица 2 – Технологии и их место в системе

Технология	Версия	Где используется
Kotlin	2.1.21	Язык всех серверных сервисов
Spring Boot / Spring Cloud Gateway	3.5.6 / 2025.0.0	Сервисы (Spring MVC, Spring JDBC, Spring Kafka), шлюз (WebFlux)
PostgreSQL	16 (alpine)	Прикладные данные и журнал аудита
Spring JDBC (NamedParameterJdbcTemplate)	–	Доступ к БД: SQL-запросы с именованными параметрами, без ORM
Liquibase	4.31.1	Миграции – отдельный одноразовый контейнер liquibase
Apache Kafka (KRaft)	3.9.1	Брокер событий: SASL/PLAIN, ACL, топики DLT
Redis	7.4	Сессии, токены обновления, коды восстановления, счётчики лимитов, идемпотентность обработчика
SeaweedFS (S3 API)	3.97	Объектное хранилище вложений
OpenLDAP (osixia)	1.5.0	Заглушка корпоративного каталога R-09
Mailpit	1.27	Заглушка почтового сервиса R-08
Nimbus JOSE+JWT	10.3	Подпись и проверка JWT (RS256)
Bouncy Castle (Argon2)	1.81	Argon2id через Argon2PasswordEncoder
nginx	1.27	Раздача SPA, TLS-прокси к шлюзу
Docker Compose	v2	Развёртывание одной командой

2.2. Обоснование замены RabbitMQ на Kafka

В ЛР № 2 брокер решал две задачи: развязать приём заявок и медленные операции (уведомления, проверку полноты моделью) и сгладить пик нагрузки в последние часы приёма (сценарий M-01). При реализации выбрана Apache Kafka, потому что для этой системы важны свойства журнала событий, которых у классической очереди нет:

– **События хранятся после прочтения.** В RabbitMQ подтверждённое сообщение удаляется; в Kafka топик `applications.events` хранит

события 7 суток. После сбоя обработчика или ошибки в нём события можно переиграть, сдвинув смещение группы, а при разборе инцидента (финансовые последствия, цель G-03) – посмотреть, что и когда происходило с заявкой.

– **Несколько независимых потребителей одного события.** Одно и то же событие о заявке нужно notifier, а в полной версии – и llm-orchestrator. В Kafka каждая группа потребителей читает топик со своим смещением; новый потребитель подключается без изменения производителя и может прочитать историю с начала. В RabbitMQ для этого нужны exchange и отдельная очередь на каждого потребителя, а история новому потребителю недоступна.
– **Порядок событий одной заявки.** Ключ сообщения – `application_id`, поэтому все события заявки попадают в одну партицию и обрабатываются в порядке возникновения: «подана» не придёт после «одобрена».
– **Пиковая нагрузка (M-01).** Топик разбит на 3 партиции: при росте нагрузки экземпляры notifier добавляются в группу и делят партиции между собой, производитель при этом не меняется.
– **Контроли B-08/B-09 сохраняются.** Каждый сервис подключается под своей учётной записью SASL; ACL разрешают производителям только запись в свой топик, notifier – только чтение своих топиков, свою группу и запись в DLT. Режим KRaft не требует ZooKeeper – на один компонент меньше.

Компромиссы, которые пришлось закрыть кодом: в Kafka нет встроенной DLQ и подтверждения отдельного сообщения. DLQ реализована топиками `*.DLT` и обработчиком ошибок Spring Kafka (3 повтора, затем перекладывание), подтверждение – фиксацией смещения после успешной обработки (ack-mode RECORD). Kafka требует больше памяти, чем RabbitMQ, что для локального стенда приемлемо.

3. Задание 1. Проект и окружение

3.1. Репозиторий и структура

Структура каталогов повторяет контейнерную диаграмму: один контейнер – один каталог с тем же именем, что и сервис в docker-compose.yml. Общий код сервисов (проверка токена, обработка ошибок, аудит, идемпотентность) вынесен в библиотеку `platform` – это не контейнер, а Gradle-модуль, который подключают сервисы.

Таблица 3 – Каталоги репозитория

Каталог	Содержимое
db/	schema.sql (из ЛР2), roles.sql, seed.sql, changes/, changelog.xml, run-migrations.sh – контейнер liquibase
api-gateway/	Шлюз: TLS, маршрутизация, проверка подписи токена, лимиты, размер тела
auth-service/	Вход, регистрация, токены, сессии, восстановление доступа
applications-service/	Заявки, вложения, кампании, подача и отзыв
review-service/	Проверка заявок, рейтинг, реестр выплат
notifier/	Потребитель Kafka, отправка писем
applicant-web/, staff-panel/	Кабинет заявителя и панель сотрудника (SPA + nginx)
kafka/, redis/, ldap/, object-storage/	Инициализация топиков и ACL, ACL Redis, данные каталога, запуск хранилища
platform/	Общая библиотека сервисов
requests/	Коллекция запросов run-all.mjs и api.http
docs/	Отчёт, журнал расхождений, карта кода, доказательства прогонов

История коммитов разбита по компонентам (каркас и секреты → БД → инфраструктура → platform → каждый сервис → клиенты → коллекция запросов → документация).

3.2. Запуск одной командой

В docker-compose.yml описаны все контейнеры диаграммы, кроме llm-orchestrator. Порядок запуска задан условиями `depends_on`: postgres (healthy) → liquibase (completed_successfully) → сервисы; kafka (healthy) → kafka-init (completed) → сервисы; шлюз ждёт Redis, клиенты – шлюз. После `docker compose up -d --build` система готова без ручных действий в БД.

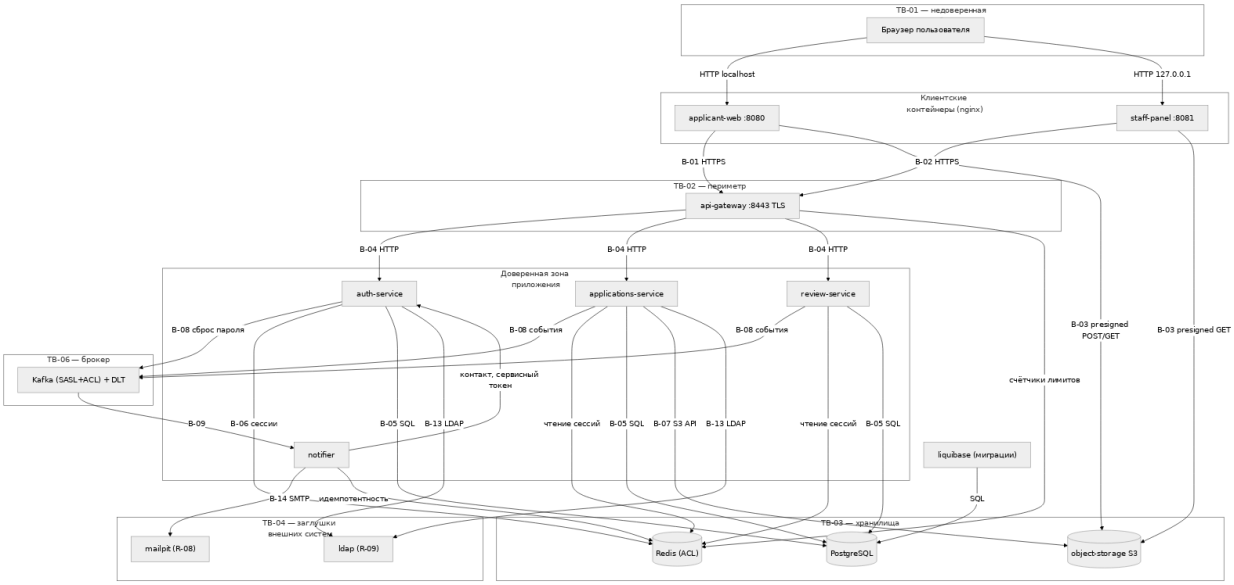


Рисунок 1 – Развёрнутые контейнеры и связи (с учётом журнала расхождений)

Наружу (только на 127.0.0.1) опубликованы: 8080 – кабинет заявителя, 8081 – панель сотрудника, 8443 – API, 8025 – Mailpit, 8090 – Kafka UI. PostgreSQL, Redis, Kafka, LDAP, хранилище и сами сервисы доступны только во внутренних сетях Docker. Сети разделены: `frontend` (nginx клиентов и шлюз, фиксированная подсеть), `backend` (сервисы и хранилища), `storage-edge` (только путь nginx → объектное хранилище).

3.3. Переменные окружения и секреты

Все настройки и секреты читаются из окружения: в коде и в application.yml нет паролей, ключей и строк подключения – только ссылки вида `${DB_PASSWORD}`. Файл `.env.example` перечисляет каждую переменную с комментарием, `.env` и каталог `secrets/` добавлены в `.gitignore`. Скрипт `scripts/gen-env.sh` генерирует случайные пароли всех учётных записей (БД, Redis, Kafka, LDAP, S3, SMTP), пару ключей RSA-2048 для подписи токенов, 32-байтный перец и самоподписанный TLS-сертификат шлюза.

Таблица 4 – Группы переменных окружения

Переменные	**Назначение**	**Кто использует**
POSTGRES_PASSWORD	Суперпользователь БД	только liquibase
SVC_*_DB_PASSWORD	Роли svc_auth, svc_applications, svc_review	сервис своей роли + liquibase
REDIS_*_PASSWORD	ACL-пользователи Redis	redis + соответствующий сервис

Переменные	Назначение	Кто использует
KAFKA_*_PASSWORD	Учётные записи SASL	kafka + сервис
JWT_PRIVATE_KEY_B64	Закрытый ключ подписи токенов	только auth-service
JWT_PUBLIC_KEY_B64	Открытый ключ проверки подписи	шлюз и все сервисы
PASSWORD_PEPPER_B64	Перец для паролей	только auth-service
LDAP_*, S3_*, SMTP_*	Учётные данные заглушек внешних систем	соответствующие сервисы
NOTIFIER_SERVICE_TOKEN	Сервисный токен notifier → auth-service	notifier, auth-service
DEMO_USERS_PASSWORD	Пароль тестовых пользователей	auth-service при первом старте

3.4. README

README содержит требования к компьютеру, команды генерации секретов, запуска, остановки и сброса базы (`docker compose down -v`), адреса и порты, тестовые учётные записи каждой роли, запуск коллекции запросов и сценарии демонстрации на защите (сообщение в брокере, падение обработчика, права учётной записи БД, журнал аудита).

4. Задание 2. База данных

4.1. Схема и миграции

База создаётся автоматически контейнером `liquibase` (образ liquibase/liquibase:4.31.1), который применяет `db/changelog.xml` и завершается. Порядок: schema.sql из ЛР № 2 без изменений → changes/001-declared-grounds.sql (D-07) → roles.sql → seed.sql (контекст demo). Liquibase ведёт таблицу databasechangelog: повторный запуск не применяет выполненные изменения, а правка уже применённого скрипта обнаруживается по контрольной сумме. ORM не используется, поэтому схема – ровно та, что в schema.sql: поля владельца (`applicant_id`, `recipient_id`), все индексы и ограничения (`uq_application_per_campaign`, `uq_payout_per_application`, `uq_protocol_per_campaign`, `ck_applications_reject` и др.) сохранены.

4.2. Учётные записи приложения

Сервисы подключаются не под postgres, а под отдельными ролями svc_auth, svc_applications, svc_review с привилегиями из п. 4.2 ЛР № 2. Права выдаются скриптом `db/roles.sql`; пароли подставляются Liquibase из окружения (changeset с `runAlways`, чтобы пароли синхронизировались с .env). Дополнительно: `REVOKE CREATE ON SCHEMA public FROM PUBLIC`, `REVOKE ALL ON DATABASE scholarship FROM PUBLIC`, атрибуты NOSUPERUSER NOCREATEDB NOCREATEROLE. Журнал `audit_log` – только INSERT для всех ролей.

Проверка на живой базе (`sh scripts/db-rights-check.sh`):

```
[svc_applications] DROP TABLE users;
    ERROR:  must be owner of table users


[svc_applications] CREATE TABLE hack (id int);
    ERROR:  permission denied for schema public
    LINE 1: CREATE TABLE hack (id int);
                          ^


[svc_applications] SELECT count(*) FROM payout_entries;
    ERROR:  permission denied for table payout_entries


[svc_applications] DELETE FROM audit_log;
    ERROR:  permission denied for table audit_log


[svc_review] UPDATE audit_log SET result = 'success';
    ERROR:  permission denied for table audit_log
```

```
[svc_review] DELETE FROM applications;
    ERROR:  permission denied for table applications

[svc_applications] SELECT count(*) AS my_visible_applications FROM applications;
     my_visible_applications
    -------------------------
                            7
    (1 row)
```

4.3. Тестовые данные

`db/seed.sql`: два подразделения (ICST и IEE), четыре заявителя (stud001 и stud002 – одно подразделение, у каждого свои заявки; stud004 отчислен по каталогу), сотрудники emp001 (ICST) и emp002 (IEE), председатели комиссии chair001 (ICST) и chair002 (IEE), служба поддержки sup001; четыре кампании (открытые ПГАС и материальная поддержка, закрытая ПГАС с одобренными заявками для реестра выплат, кампания IEE); критерии и баллы. Окна приёма считаются от момента миграции (`now() ± interval`), поэтому сценарии не зависят от даты защиты.

Пароли тестовых пользователей тоже хранятся хешем. В seed.sql вместо хеша стоит маркер `!bootstrap` (не совпадает ни с одним паролем – проверка принимает только строки `$argon2id$`). При первом старте auth-service вычисляет Argon2id(перец, DEMO_USERS_PASSWORD) для таких записей; готовый хеш в репозитории потребовал бы зафиксировать перец (D-10). После запуска в базе:

```
scholarship=# SELECT external_id, left(password_hash, 40) FROM users LIMIT 2;
 stud001 | $argon2id$v=19$m=65536,t=3,p=4$Lq0Y9Zb...
 stud002 | $argon2id$v=19$m=65536,t=3,p=4$8rKcV1o...
```

5. Задание 3. Аутентификация и токены

5.1. Регистрация и вход

Пароль хешируется функцией из п. 4.3 ЛР № 2: **Argon2id, память 64 МБ, 3 итерации, параллелизм 4**, соль 16 байт уникальна для пользователя и хранится внутри строки PHC в `users.password_hash`. Перец (32 байта из `PASSWORD_PEPPER_B64`, в базе отсутствует) примешивается как HMAC-SHA256(перец, пароль) до вызова Argon2id (`auth-service/.../crypto/PasswordHasher.kt`).

Неверный логин, неверный пароль и заблокированная запись дают одинаковый ответ `401 {"error":"invalid_credentials","message":"Неверный логин или пароль"}`. Время ответа тоже выровнено: если пользователя нет, Argon2id всё равно выполняется – против заранее вычисленного хеша-заглушки. После проверки пароля учётная запись сверяется с каталогом R-09 (B-13).

Регистрация (OP-14, D-13) доступна только действующему обучающемуся: логин и e-mail сверяются с каталогом LDAP, ФИО и подразделение берутся из каталога, роль `applicant` назначает сервер. Лишнее поле `role` в теле даёт 400.

5.2. Политика токенов

Таблица 5 – Политика токенов

Параметр	Токен доступа	Токен обновления
Тип	JWT, подпись RS256 (typ = at+jwt)	Случайные 256 бит (не JWT)
Хранение у клиента	Только в памяти JS (переменная модуля api.js), не localStorage	Cookie HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth
Хранение на сервере	Не хранится; сессия `session:{sid}` в Redis	Только SHA-256 в Redis (`refresh:{hash}`)
Время жизни	10 минут	До конца сессии (8 часов, абсолютный срок)
Полезная нагрузка	iss, aud, sub, sid, jti, roles, unit_id, iat, nbf, exp – без ФИО и e-mail	–
Отзыв	Удаление сессии в Redis: каждый сервис проверяет session:{sid} → 401	Ротация при каждом обновлении; повторное предъявление старого токена отзывает всю сессию

Параметр	Токен доступа	Токен обновления
Подпись / проверка	Закрытый ключ только у auth-service; шлюз и сервисы проверяют открытым	–

Проверка токена (`platform/.../token/AccessTokenVerifier.kt`) выполняется дважды: на шлюзе (запрос с негодным токеном не доходит до сервисов) и в каждом сервисе заново (`AuthenticationFilter`, граница ТВ-02 – «сервис не доверяет шлюзу»). Алгоритм строго RS256 (токены с `alg=none` и HS256 отклоняются), проверяются тип, подпись, iss, aud, exp и nbf с допуском 30 с, затем – наличие сессии в Redis. Результат на живой системе:

```
[OK]   401 без токена
[OK]   401 просроченный токен (верная подпись, exp в прошлом)
[OK]   401 подделанная подпись
[OK]   401 изменённая полезная нагрузка (роль) с прежней подписью
[OK]   401 alg=none
[OK]   204 выход
[OK]   401 ПОСЛЕ ВЫХОДА токен отклоняется (отозван)
[OK]   401 повторное использование старого refresh
[OK]   401 после обнаружения повтора вся сессия отозвана
```

5.3. Восстановление доступа

1. `POST /auth/password-reset {email}` – ответ всегда `202` с одинаковым текстом, существует адрес или нет. Для существующего пользователя создаётся код: 256 бит, в Redis хранится только его SHA-256 (`pwreset:{hash}`, TTL 15 минут); предыдущий неиспользованный код аннулируется.
2. auth-service публикует событие в топик `auth.password-reset` (срок хранения сообщений = сроку кода, 15 минут); notifier получает адрес у auth-service и отправляет письмо со ссылкой в Mailpit.
3. `POST /auth/password-reset/confirm {token, new_password}` – код извлекается атомарной командой GETDEL, поэтому срабатывает ровно один раз даже при параллельных запросах; пароль проверяется политикой (10–128 символов, не содержит логин), все прежние сессии пользователя отзываются.

```
[OK]   202 запрос для существующего адреса → 202
[OK]   202 запрос для несуществующего адреса → такой же 202
[OK]   письмо с одноразовой ссылкой пришло
[OK]   204 смена пароля по коду
[OK]   400 код срабатывает один раз → 400
[OK]   401 старые сессии завершены после смены пароля → 401
```

```
[OK]   401 вход со старым паролем → 401
```

6. Задание 4. Эндпоинты по контракту

6.1. Объём

Реализованы все операции из п. 3.1 ЛР № 2, кроме операции с языковой моделью (OP-06/OP-11 – заглушка 501), и операции, добавленные по журналу расхождений.

Таблица 6 – Реализованные операции

ID	Метод и путь	Сервис	Коды ответа
OP-01	GET /applications/{id}	applications	200, 401, 404, 429
OP-02	POST /applications	applications	201, 200 (повтор), 400, 401, 403, 409, 422, 429
OP-03	PATCH /applications/{id}/review	review	200, 400, 403, 404, 409, 422, 428
OP-04	GET /applications	applications	200, 400, 401, 429
OP-05	POST /campaigns/{id}/payout-registry	review	201, 200 (повтор), 403, 409, 422, 428
OP-06/11	POST/GET /applications/{id}/completeness-check	applications	501
OP-07	POST /auth/login	auth	200, 400, 401, 429
OP-08	POST /auth/refresh	auth	200, 401
OP-09	POST /auth/password-reset (+ /confirm)	auth	202; 204, 400
OP-10	POST /applications/{id}/attachments	applications	201, 400, 404, 409, 422
OP-12	GET /campaigns/{id}/ranking	review	200, 403, 404
OP-13	DELETE /applications/{id}	applications	204, 404, 409, 422
OP-14…16	POST /auth/register, /logout, /reauth	auth	201/204, 400, 403, 409, 422
OP-17	GET /campaigns	applications	200
OP-18	POST /applications/{id}/submit	applications	200, 404, 409, 422

6.2. Соответствие контракту

Ответ каждой операции – отдельный класс DTO ровно с полями из таблиц ЛР № 2; строка таблицы клиенту не отдаётся, хеш пароля, content_hash, version и служебные поля не возвращаются (версия передаётся заголовком ETag, как требует If-Match в OP-03). Тела запросов – тоже DTO с закрытым списком полей, а в Jackson включено `fail-on-unknown-properties: true`: любое лишнее поле (applicant_id, status, reviewed_by, checksum…) даёт `400`. В OP-01 и OP-04 лишние query-параметры игнорируются: «посмотреть от имени другого» не имеет синтаксиса.

6.3. Порядок вызовов

Шаги внутри обработчиков идут в порядке диаграмм последовательности ЛР № 2. Для OP-01 (рисунок 4 ЛР № 2): фильтр проверяет токен и сессию → контроллер проверяет право роли `applications:read` → выборка с проверкой принадлежности **в том же SQL-запросе** → запись в журнал → ответ. Чужая и несуществующая заявки неразличимы (оба – ноль строк → 404):

```
-- ApplicationRepository.findVisible
SELECT a.id, a.applicant_id, ... FROM applications a
JOIN campaigns c ON c.id = a.campaign_id
WHERE a.id = :id
  AND (   (:own  AND a.applicant_id = :sub)
       OR (:unit AND c.unit_id = (SELECT u.unit_id FROM users u WHERE u.id = :sub)) )
```

Для OP-05 (рисунок 5 ЛР № 2): роль committee_chair → Idempotency-Key → повторный ввод пароля не старше 5 минут → схема тела → поиск сохранённого ответа по ключу → BEGIN → кампания закреплена за комиссией → реестра ещё нет → каждая заявка approved и из этой кампании, сумма в границах → INSERT protocols → INSERT payout_entries (получатель – из заявки в БД) → INSERT audit_log → сохранение ключа → COMMIT. Проверка прав всегда предшествует возврату и изменению данных.

6.4. Повторная отправка

Таблица 7 – Поведение при повторе неидемпотентных операций

Операция	Реализация	Проверено
OP-02	Idempotency-Key (таблица idempotency_keys, ключ в пределах субъекта и эндпоинта, advisory-блокировка на ключ); второй рубеж – uq_application_per_campaign	повтор → 200 и тот же id; другой ключ → 409; тот же ключ с другим телом → 422
OP-03	If-Match с версией; UPDATE ... WHERE version = :v; без заголовка – 428	устаревшая версия → 409; повтор того же PATCH → 409
OP-05	Три рубежа: Idempotency-Key → проверка «реестра нет» → ограничения БД uq_protocol_per_campaign, uq_payout_per_application	повтор → 200 и тот же registry_id; новый ключ → 409; прямой INSERT в БД → ошибка ограничения
OP-13, OP-18	Проверка текущего статуса: повторная подача и повторный отзыв возвращают тот же результат	повторы → 200 / 204

6.5. Обработка ошибок

Единый обработчик `platform/.../web/ApiErrorHandler.kt`: ожидаемые ошибки возвращают код из контракта и нейтральное сообщение, непредвиденные – `500 {"error":"internal_error","message":"Внутренняя ошибка сервиса","request_id":…}`. Текст исключения, стек и SQL пишутся только в журнал контейнера вместе с request_id; в Spring Boot отключены `include-message`, `include-stacktrace`. Все запросы к базе параметризованы (`:name` в NamedParameterJdbcTemplate). Сортировка OP-04 выбирается из белого списка готовых SQL-фрагментов (enum ListSort) – значение параметра `sort` в текст запроса не попадает (`sort=id;DROP TABLE users` → 400). Поиск в LDAP также параметризован: фильтр `(uid={0})` с экранированием аргумента.

7. Задание 5. Межсервисное взаимодействие

7.1. Брокер Kafka

Таблица 8 – Топики и права

Топик	Партиции / хранение	Пишут	Читают
applications.events	3 / 7 суток	applications-service (submitted, withdrawn), review-service (status_changed)	notifier (группа notifier)
auth.password-reset	1 / 15 минут	auth-service	notifier
*.DLT	1 / 30 суток и 15 минут	notifier (перекладывание)	разбор вручную, Kafka UI

Сообщение содержит только идентификаторы (B-08): `{event_id, type, application_id, campaign_id, recipient_id, status, occurred_at}`, ключ – application_id. Публикация выполняется после фиксации транзакции (`TransactionSynchronization.afterCommit`), поэтому в брокер не попадает событие об откаченном изменении. Демонстрация: сообщение в топике и письмо, отправленное по нему notifier.

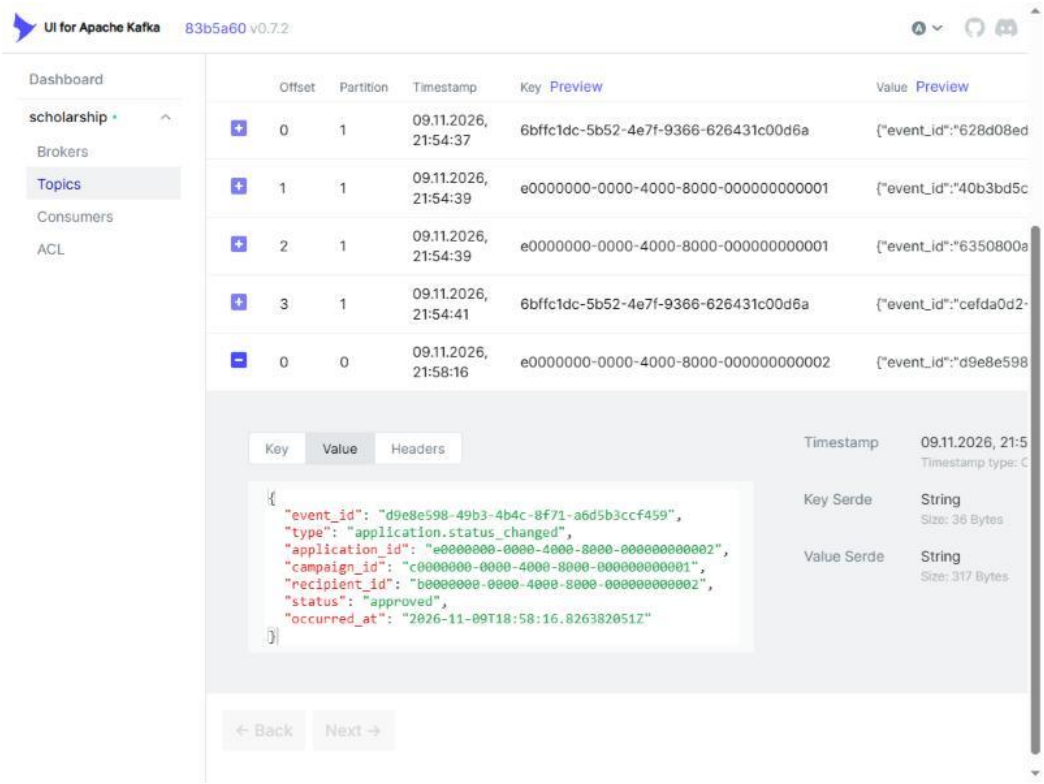


Рисунок 2 – Сообщение в топике applications.events (Kafka UI)

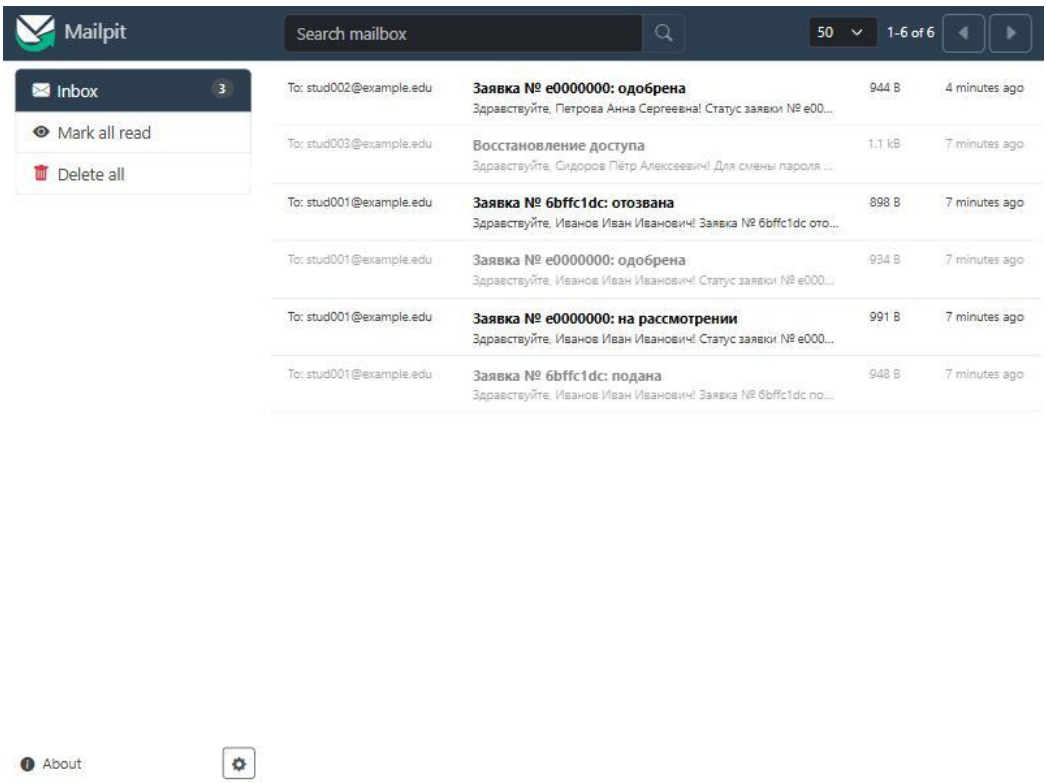


Рисунок 3 – Результат обработки: письма в почтовой заглушке

Падение обработчика. notifier фиксирует смещение только после успешной отправки письма (ack-mode RECORD, автофиксация выключена). Если notifier остановлен или упал во время обработки, сообщения остаются в топике (хранение 7 суток), отставание группы видно в Kafka UI; после запуска notifier продолжает с последнего зафиксированного смещения. Временная ошибка (недоступна почта или auth-service) – 3 повтора с паузой 2 с, затем сообщение перекладывается в `<топик>.DLT`, и очередь не блокируется; неразбираемое сообщение уходит в DLT сразу.

Повторная доставка. Гарантия – «хотя бы один раз», поэтому обработчик идемпотентен: после отправки письма `event_id` записывается в Redis (`notifier:done:{event_id}`, TTL 7 суток), повторно доставленное событие пропускается с записью в журнал. Остаётся узкое окно (сбой между отправкой письма и записью отметки), в котором возможно второе письмо; для уведомлений это допустимо.

Падение брокера. Основная операция (создание, проверка заявки) уже зафиксирована в БД и завершается успешно; публикация завершается ошибкой через 5 с ожидания и записывается в журнал, уведомление теряется.

Это осознанный компромисс: уведомление – не источник истины, статус всегда виден в кабинете. Полное решение – паттерн transactional outbox – потребовало бы таблицы, которой нет в schema.sql.

7.2. Redis

Таблица 9 – Использование Redis

Ключ	**Кто пишет / читает**	**TTL**	**Когда сбрасывается**
session:{sid}	auth (запись); gateway – нет; applications, review – только чтение (ACL %R~)	8 ч	выход, обнаружение повторного refresh, смена пароля
refresh:{sha256}, refresh-used:{sha256}	auth	остаток сессии	ротация (GETDEL), отзыв сессии
user-sessions:{user_id}	auth	8 ч	смена пароля (отзыв всех сессий)
pwreset:{sha256}, pwreset-user:{id}	auth	15 мин	использование кода (GETDEL), новый запрос
rl:{правило}:{субъект|ip}:{окно}	api-gateway	окно + 5 с	истечение окна
enum404:{user_id}	applications (серия 404 – событие перебора)	1 мин	истечение
notifier:done:{event_id}	notifier	7 суток	истечение

Кэша персональных данных в системе нет (кэш отчётов модели относился к llm-orchestrator); все ключи, связанные с пользователем, содержат его идентификатор или сессию. ACL Redis: пользователь `default` выключен, каждый клиент видит только свой префикс, опасные команды (FLUSHALL, KEYS, CONFIG) запрещены. Если Redis недоступен, сервисы отказывают в доступе (503, fail closed), а шлюз пропускает запросы без учёта лимита – лимит защищает от перегрузки, а не управляет доступом.

7.3. Синхронные вызовы

– **Шлюз → сервисы (B-04).** Шлюз передаёт исходный токен и не добавляет заголовков с ролью; входящие X-User-*, X-Service-Token и X-Request-Id удаляются. Сервис проверяет токен сам.

– **notifier → auth-service** (`GET /internal/users/{id}/contact`, D-15). Вызываемый сервис проверяет вызывающего: заголовок X-Service-Token сравнивается с секретом за постоянное время, иначе 403. Путь

/internal не маршрутизируется шлюзом, порт auth-service не опубликован.

– **Сервисы → LDAP, PostgreSQL, S3** – под своими учётными записями с минимальными правами (каталог – только чтение; S3 – только бакет вложений).

8. Задание 6. Клиентское приложение

Как и на диаграмме ЛР № 2, клиентов два: кабинет заявителя (applicant-web, http://localhost:8080) и панель сотрудника (staff-panel, http://127.0.0.1:8081). Это SPA на JavaScript без сборки, раздаваемые nginx; nginx проксирует `/api/` на шлюз по HTTPS с проверкой его сертификата и `/applications-attachments/` – на объектное хранилище. Клиенты обращаются только к эндпоинтам контракта; доступа к базе нет. Панель открывается по 127.0.0.1, потому что cookie не различают порты: другой хост разделяет сессии двух приложений.

- **Заявитель:** регистрация, вход, восстановление пароля по ссылке из письма, список своих заявок с фильтром, создание (ключ идемпотентности генерируется при открытии формы), карточка, загрузка вложения напрямую в хранилище, подача, отзыв, кнопка проверки полноты (ответ 501).
- **Сотрудник:** список заявок подразделения с фильтрами, сортировкой и курсором, карточка с вложениями и баллами, форма проверки (статус, баллы по критериям, причина; If-Match из ETag), рейтинг.
- **Председатель комиссии:** рейтинг и формирование реестра выплат с повторным вводом пароля; результат – число строк, сумма и SHA-256.
- **Служба поддержки:** просмотр заявки по номеру без вложений и баллов.

Токен доступа хранится только в памяти (раздел 5.2); после перезагрузки страницы сессия восстанавливается через /auth/refresh. Проверки в интерфейсе (скрытые пункты меню по ролям, maxlength, обязательные поля) – удобство: каждая продублирована на сервере, что подтверждают запросы коллекции «минуя интерфейс». CSP запрещает встроенные скрипты, DOM строится только через textContent; загруженные пользователями файлы отдаются с `Content-Disposition: attachment` и `CSP: sandbox`.

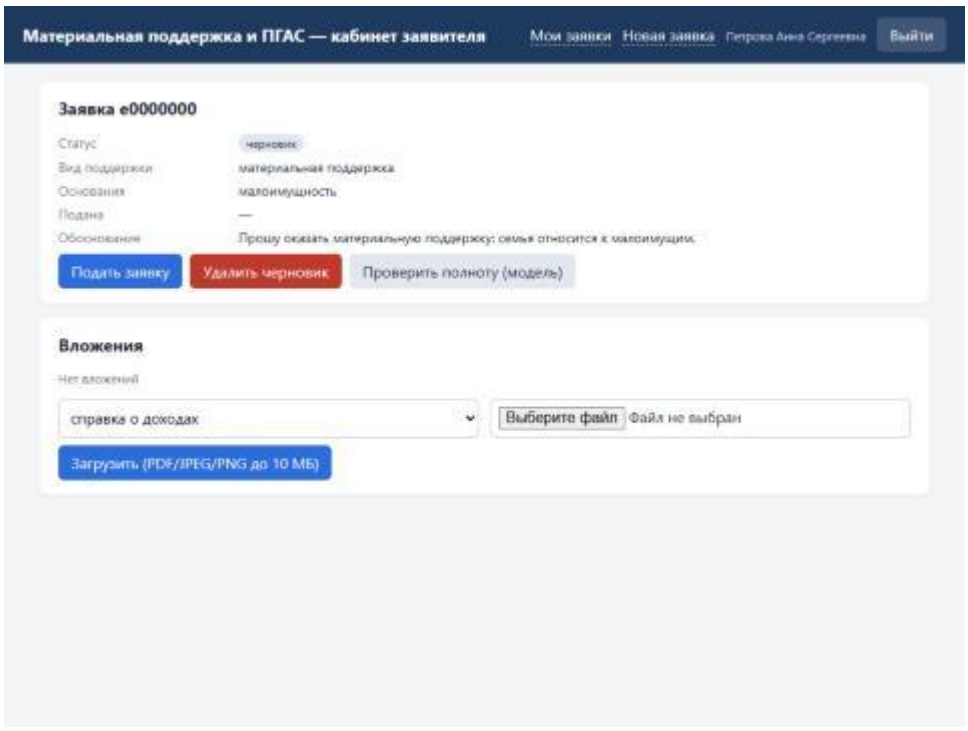


Рисунок 4 – Кабинет заявителя: карточка черновика и загрузка вложения

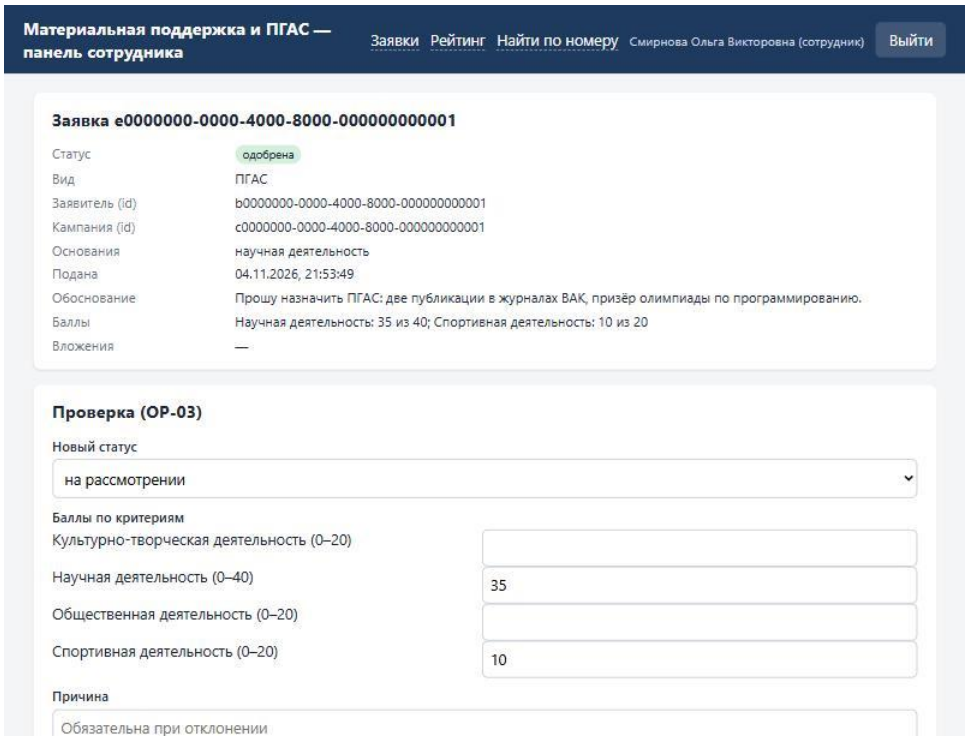


Рисунок 5 – Панель сотрудника: карточка заявки и форма проверки (ОР-03)

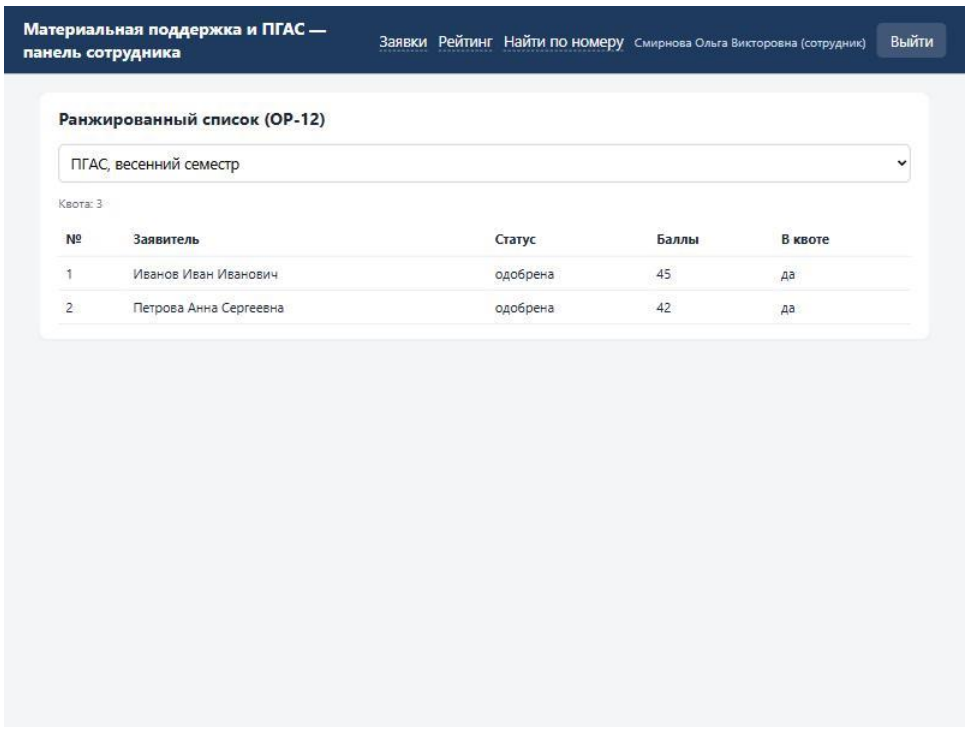


Рисунок 6 – Панель сотрудника: ранжированный список (ОР-12)

9. Задание 7. Проверка и карта кода

9.1. Коллекция запросов

В каталоге requests/ два формата: `run-all.mjs` – сценарий на Node.js (встроенный fetch), который проходит по всем эндпоинтам на тестовых данных и сверяет фактический код с ожидаемым, и `api.http` – ручные запросы для HTTP-клиента IntelliJ IDEA / VS Code. Сценарий также проверяет строки в БД, письма в Mailpit и права ролей PostgreSQL. Полный вывод последнего прогона – `docs/evidence/run-all.log`; итог: **успешно 149, ошибок 0**.

```
$ NODE_EXTRA_CA_CERTS=secrets/gateway.crt node requests/run-all.mjs
...
ИТОГО: успешно 149, ошибок 0
```

9.2. Три проверочных запроса

Проверка 1. Запрос объекта другого пользователя с действительным токеном. Ожидаемый код по ЛР № 2 – 404.

```
GET /api/v1/applications/e0000000-0000-4000-8000-000000000002     (заявка stud002)
Authorization: Bearer <токен stud001>


HTTP/1.1 404
{"error":"not_found","message":"Объект не найден","request_id":"cfa88d78-…"}
```

Отбил запрос SQL выборки (`ApplicationRepository.findVisible`, раздел 6.3): условие `a.applicant_id = :sub`, где `:sub` – идентификатор из проверенного токена, а не из запроса. Строк ноль → `ApplicationService.get` пишет отказ в audit_log и бросает `Errors.notFound()`; ответ совпадает с ответом на несуществующий id.

```
val row = repo.findVisible(id, user.id, own, unit)
if (row == null) {
    audit.write("application.view", "application", id, user.id, AuditResult.DENIED)
    countNotFound(user)          // серия 404 – событие перебора
    throw Errors.notFound()
}
```

Проверка 2. Создание объекта с лишними полями: статус, идентификатор владельца, признак роли. Ожидаемый код по ЛР № 2 – 400 («любое лишнее поле даёт 400, а не молчаливо игнорируется»).

```
POST /api/v1/applications
Authorization: Bearer <токен stud002>
Idempotency-Key: <uuid>
{"campaign_id":"c0000000-…-
004","type":"pgas","statement":"…","declared_grounds":["science"],
 "status":"approved","applicant_id":"b0000000-…-001","role":"committee_chair"}
```

```
HTTP/1.1 400
{"error":"invalid_request","message":"Недопустимое поле в теле запроса:
status","request_id":"193cd72c-…"}
```

Отбила десериализация: DTO содержит только четыре поля контракта, а Jackson настроен с `fail-on-unknown-properties: true`; исключение UnrecognizedPropertyException превращается в 400 в ApiErrorHandler. До обработчика запрос не дошёл, строка не создана.

```
data class CreateApplicationRequest(
    val campaignId: String?, val type: String?, val statement: String?, val declaredGrounds:
List<String>?,
)
// application.yml:  spring.jackson.deserialization.fail-on-unknown-properties: true
```

Строка в БД для заявки, созданной корректным запросом того же сценария, – владелец взят из токена, статус назначен схемой (`DEFAULT 'draft'`), а не прислан клиентом:

```
scholarship=# SELECT applicant_id, status FROM applications WHERE id = '<id созданной
заявки>';
 b0000000-0000-4000-8000-000000000001 | draft
-- попытка с лишними полями строк не добавила:
scholarship=# SELECT count(*) FROM applications WHERE campaign_id='c0000000-…-004' AND
applicant_id='b0000000-…-002';
 0
```

Проверка 3. Вызов привилегированной операции токеном обычного пользователя, минуя интерфейс. Ожидаемый код по ЛР № 2 – 403.

```
POST /api/v1/campaigns/c0000000-0000-4000-8000-000000000003/payout-registry
Authorization: Bearer <токен stud001>
Idempotency-Key: <uuid>
{"protocol_number":"П-15","protocol_date":"2026-11-09","entries":[…]}

HTTP/1.1 403
{"error":"forbidden","message":"Операция доступна только председателю
комиссии","request_id":"bad30071-…"}
```

Отбила проверка права на операцию в контроллере – право `payout:create` есть только у роли committee_chair (матрица `RolePermissions`), роли берутся из подписанного токена. Сотрудник (reviewer) своей кампании получает тот же 403.

```
// ReviewController.registry
if (!user.has(Permission.PAYOUT_CREATE)) throw Errors.forbidden("Операция доступна только
председателю комиссии")
// Permissions.kt
Roles.COMMITTEE_CHAIR to setOf(..., Permission.PAYOUT_CREATE, ...)
```

9.3. Карта кода

Соответствие проектных решений и файлов кода приведено в `docs/code-map.md`. Основное:

Решение	**Где в коде**
Проверка подписи и срока токена; ключ	platform/token/AccessTokenVerifier.kt; открытый ключ – JWT_PUBLIC_KEY_B64, закрытый – только в auth-service
Идентификатор текущего пользователя	platform/web/Filters.kt (AuthenticationFilter) → AuthUserArgumentResolver; никогда из тела
Различие доступа ролей	platform/security/Permissions.kt (на операцию) + WHERE в репозиториях (на объект)
Хеширование паролей, перец	auth-service/crypto/PasswordHasher.kt
Повторная доставка сообщения	notifier/NotificationListener.kt (process, notifier:done:{event_id})
Граница доверия ТВ-02	api-gateway/GatewayFilters.kt (JwtFilter) + AuthenticationFilter в каждом сервисе
Журнал аудита	platform/audit/AuditLog.kt (success – в транзакции, denied – REQUIRES_NEW)

10. Пошаговое выполнение работы

1. **Анализ ЛР № 2.** Выписаны все контейнеры, связи B-01…B-16, операции OP-01…OP-13, поля и коды контрактов, GRANT из schema.sql. Обнаружены пробелы проекта (нет перехода draft→submitted, нет хранения declared_grounds, нет справочника кампаний, не описана политика токенов) – они стали строками журнала расхождений.
2. **Каркас репозитория.** `git init`, .gitignore (исключены .env и secrets/), .env.example с описанием каждой переменной, скрипт gen-env.sh (openssl: случайные пароли, RSA-2048, перец, TLS-сертификат шлюза).
3. **База данных.** schema.sql скопирован из ЛР № 2; блок ролей вынесен в roles.sql с параметрами Liquibase; написаны миграция declared_grounds, seed.sql и changelog.xml; контейнер liquibase добавлен в compose. Проверено: миграции применяются на чистой базе, повторный запуск ничего не меняет.
4. **Инфраструктура.** Kafka в режиме KRaft с SASL/PLAIN и ACL (kafka-init создаёт топики, DLT и права), Redis с ACL-файлом, OpenLDAP с данными каталога, SeaweedFS с учётными записями S3, Mailpit. Проверены ACL: сервис applications не может записать ключ session:*, не может выполнить KEYS.
5. **Общая библиотека platform.** Проверка JWT, фильтр аутентификации с проверкой сессии, единый обработчик ошибок, аудит, ключи идемпотентности, публикация событий после фиксации транзакции.
6. **Первый эндпоинт целиком.** auth-service (вход, выпуск токена) и OP-01 в applications-service – от запроса через шлюз до SQL с проверкой владельца и записи в audit_log. Проверено: своя заявка 200, чужая 404, неверный логин и пароль – одинаковый 401.
7. **Остальные операции** по тому же образцу: OP-02, OP-04, OP-10, OP-13, OP-17, OP-18 (applications-service); OP-03, OP-05, OP-12 (review-service); регистрация, обновление с ротацией, выход, повторный ввод пароля, восстановление доступа (auth-service).

8. **Асинхронная обработка:** notifier – потребитель Kafka с идемпотентностью и DLT, отправка писем в Mailpit. Найдена и исправлена ошибка: RestClient, созданный вручную, не использовал snake_case – сообщения уходили в DLT (причина видна в журнале обработчика).
9. **Шлюз:** TLS, маршруты, проверка подписи, лимиты частоты в Redis, ограничение размера тела, удаление заголовков-«представлений».
10. **Клиенты:** кабинет заявителя и панель сотрудника; проверка сценариев каждой роли в браузере. Найдено и исправлено: cookie обновления общая для localhost:8080 и localhost:8081 – панель вынесена на 127.0.0.1.
11. **Коллекция запросов** run-all.mjs: все эндпоинты, три проверочных запроса, проверки БД и почты. Первый прогон выявил превышение лимита OP-05 (5/ч) самим сценарием – проверки распределены между двумя председателями, лимит контракта не ослаблен. Итоговый прогон на чистой базе: 149 из 149.
12. **Документация:** README, журнал расхождений, карта кода, отчёт.

Команды полного цикла проверки:

```
sh scripts/gen-env.sh
docker compose up -d --build
NODE_EXTRA_CA_CERTS=secrets/gateway.crt node requests/run-all.mjs
sh scripts/db-rights-check.sh
docker compose down -v
```

11. Выводы

Система, спроектированная в ЛР № 2, реализована и разворачивается одной командой: 15 контейнеров, из них 5 сервисов на Kotlin/Spring, 2 клиентских приложения, PostgreSQL с миграциями Liquibase, Kafka, Redis, объектное хранилище и заглушки внешних систем. Все операции контракта, кроме операции с языковой моделью, реализованы с путями, полями и кодами ответа из ЛР № 2; 149 автоматических проверок на живой системе проходят.

Решения по безопасности выражены в коде: владелец объекта проверяется в самом SQL-запросе; субъект берётся только из подписанного токена, который каждый сервис проверяет заново и сверяет с сессией в Redis, поэтому выход действительно отзывает токен; лишние поля в теле отклоняются; повторная отправка закрыта ключами идемпотентности, версией в If-Match и ограничениями БД; пароли хранятся как Argon2id с солью и перцем вне базы; каждый сервис ходит в БД, Redis, Kafka и S3 под своей учётной записью с минимальными правами, журнал аудита можно только дополнять.

Реализация показала, где проект ЛР № 2 был неполон (22 строки журнала расхождений): часть пробелов – функциональные (переход draft→submitted, хранение оснований, справочник кампаний), часть – уточнения безопасности (политика токенов, механизм повторного подтверждения личности, источник адреса для уведомлений без ПДн в брокере). Замена RabbitMQ на Kafka сохранила контроли B-08/B-09 и дала хранимый журнал событий с повторным чтением. Код из этой работы – объект проверки в ЛР № 4.